

Simulado — não vale nota

Disciplina: Banco de Dados II

Semestre: 2026.2

Professor: Alysson M. Bruno

Tema: Revisão para a A1 (Aulas 01 a 06)

Turma no Google Classroom: <https://classroom.google.com/c/ODY5MzE1NDkyODUx>

Como fazer

- Responda **sem consulta**, como na prova: sem o roteiro de revisão, sem os slides e sem internet.
- Parte I: 10 questões objetivas, com quatro alternativas (A, B, C e D) e apenas uma correta. Anote a letra de cada uma num papel. Tempo sugerido: 20 minutos.
- Parte II: 1 questão discursiva. **Escreva a resposta à mão, no papel, com caneta** — a prova é manuscrita, e é assim que se treina para ela. Tempo sugerido: 20 minutos.
- Só depois de terminar, confira as respostas no capítulo **Resolução comentada do simulado**, no fim do roteiro de revisão.
- Os casos são hipotéticos, criados para este simulado.

Quadro de respostas (Parte I)

Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10

Questão 1

Um estacionamento vai trocar o caderno de anotações por um banco de dados, com as tabelas mensalista, veiculo e permanencia (uma linha para cada vez que um veículo entra e sai). O dono ouviu de um conhecido que “banco NoSQL é sempre melhor” e que “o MySQL nem tem controle de versões das linhas”.

Considerando as informações apresentadas, avalie as afirmações a seguir.

I. Em permanencia, cada linha é uma tupla e hora_entrada é um atributo; o domínio desse atributo inclui a regra do negócio, e não apenas o tipo de dado.

II. Um banco NoSQL é sempre mais escalável que um relacional e, por isso, deve ser a escolha padrão de todo sistema novo.

III. Se permanencia.veiculo_id for chave estrangeira para veiculo, o SGBD recusa o registro de uma permanência com um veículo que não existe.

IV. O MySQL não tem MVCC, o que o torna inadequado para um sistema com vários caixas gravando ao mesmo tempo.

É correto apenas o que se afirma em

A I e II.

B I e III.

C II e IV.

D I, III e IV.

Questão 2

Uma locadora de bicicletas registra cada aluguel em uma única tabela. Regra de negócio: o identificador da bicicleta determina o modelo e o tamanho do aro.

```
aluguel(id_aluguel, retirada, cliente_cpf, bicicleta_id, bicicleta_modelo,
bicicleta_aro)
chave primária: id_aluguel
```

Considerando as regras de negócio e a tabela apresentada, assinale a opção correta.

A Está na 3FN, pois todas as colunas dependem de id_aluguel; repetir o modelo da bicicleta em cada aluguel é só um custo de espaço.

B Viola a 2FN, porque bicicleta_modelo depende apenas de parte da chave primária id_aluguel.

C Viola a 1FN, porque a mesma bicicleta aparece em vários aluguéis; a correção é permitir um único aluguel por bicicleta.

D Viola a 3FN: modelo e aro dependem de bicicleta_id, que não é chave; devem ir para uma tabela bicicleta.

Questão 3

Uma academia vai cadastrar os seus planos. A tabela plano acabou de ser criada e está vazia.

```
CREATE TABLE plano (
  id_plano integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nome      varchar(40) NOT NULL UNIQUE,
  meses     integer     NOT NULL CHECK (meses BETWEEN 1 AND 12),
  valor     numeric(8,2) NOT NULL CHECK (valor > 0)
);

INSERT INTO plano (nome, meses, valor) VALUES
('Mensal', 1, 119.90),
('Semestral', 6, 599.00),
('Bienal', 24, 1990.00);
```

Com base na definição da tabela e no comando executado, assinale a opção que descreve corretamente o resultado.

A O comando é recusado por inteiro: a terceira linha viola o CHECK de meses, e nenhuma das três linhas é gravada.

B As duas primeiras linhas são gravadas e a terceira é descartada, porque cada linha de um INSERT é conferida e gravada separadamente.

C As três linhas são gravadas, e o valor 24 de meses é trocado por 12, o maior valor aceito pela restrição CHECK.

D O comando é recusado porque id_plano não foi informado, e a chave primária não pode ficar sem valor.

Questão 4

Uma academia vai reorganizar as aulas e pediu **uma única lista** com todas as salas e todos os instrutores lado a lado: salas ainda sem instrutor e instrutores ainda sem sala **também devem aparecer**. Tabelas: sala(id, nome) e instrutor(id, nome, sala_id), em que sala_id pode ficar nulo.

Considerando o pedido e as tabelas, assinale a alternativa que apresenta a consulta que atende ao relatório.

A `SELECT s.nome AS sala, i.nome AS instrutor FROM sala s LEFT JOIN instrutor i ON i.sala_id = s.id`

B `SELECT s.nome AS sala, i.nome AS instrutor FROM sala s RIGHT JOIN instrutor i ON i.sala_id = s.id`

C `SELECT s.nome AS sala, i.nome AS instrutor FROM sala s FULL OUTER JOIN instrutor i ON i.sala_id = s.id`

D `SELECT s.nome AS sala, i.nome AS instrutor FROM sala s INNER JOIN instrutor i ON i.sala_id = s.id`

Questão 5

O servidor PostgreSQL de uma rede de estacionamento desligou num pico de energia. Ao ser religado, voltou a aceitar conexões em poucos segundos, sem precisar reprocessar todo o registro de alterações acumulado desde a instalação.

Considerando o contexto apresentado, avalie as asserções a seguir e a relação proposta entre elas.

I. Na recuperação após a falha, o servidor reprocessa apenas o trecho do WAL posterior ao último ponto de sincronização, e não todo o WAL desde a instalação.

PORQUE

II. Os checkpoints marcam pontos de sincronização das páginas de dados e limitam o trecho de WAL que precisa ser processado durante a recuperação.

A respeito dessas asserções, assinale a opção correta.

A As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.

B As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.

C A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.

D A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.

Questão 6

Uma rede de lavanderias tem três unidades. Na proposta da equipe, o servidor de cada unidade guarda apenas as linhas de `pedido_lavagem` daquela unidade, com todas as colunas. O relatório da diretoria continua escrevendo `SELECT ... FROM pedido_lavagem`, sem indicar em qual servidor está cada linha, e o sistema gerenciador se encarrega de reunir as partes.

Considerando as informações apresentadas, assinale a alternativa que classifica corretamente a proposta.

A Fragmentação vertical, pois cada unidade guarda um grupo de colunas, e o relatório reconstrói a tabela por junção pela chave.

B Replicação, pois cada unidade guarda uma cópia completa de `pedido_lavagem`, e o relatório escolhe a cópia mais próxima.

C Arquitetura centralizada, pois o relatório da diretoria enxerga uma única tabela e, portanto, os dados estão em um só servidor.

D Fragmentação horizontal por unidade, com transparência de localização e de fragmentação para quem consulta a tabela lógica.

Questão 7

Um pet shop tem a função `fn_preco_banho(p_porte text) RETURNS numeric`, em PL/pgSQL, que lê o preço na tabela `preco_banho(porte, valor)` com `SELECT valor INTO v_valor ...` e o devolve. A equipe quer que ela passe a devolver um texto, como `'R$ 60.00'`, e executou de novo o mesmo `CREATE OR REPLACE FUNCTION`, trocando apenas `RETURNS numeric` por `RETURNS text`.

Considerando o cenário e o que foi visto sobre rotinas em PL/pgSQL, avalie as afirmações a seguir.

I. O servidor recusa o comando: `CREATE OR REPLACE` não troca o tipo de retorno de uma função existente, e é preciso removê-la com `DROP FUNCTION` antes de criá-la de novo.

II. Como a função lê uma tabela, a classificação de volatilidade adequada é `STABLE`; `IMMUTABLE` é para funções que dependem apenas dos argumentos.

III. O código entre os `$$` fica guardado em um arquivo no computador de quem criou a função e deixa de funcionar se esse arquivo for apagado.

IV. Se o porte informado não existir em `preco_banho`, o `SELECT ... INTO` provoca erro, e a função não devolve nada.

É correto apenas o que se afirma em

A I, II e IV.

B III e IV.

C I e II.

D II e III.

Questão 8

Um clube de futsal quer guardar em `atleta_arquivo` os dados de cada atleta excluído. Depois de criar o gatilho abaixo, o comando `DELETE FROM atleta WHERE id = 9;` respondeu `DELETE 0: a atleta 9` continuou na tabela `atleta`, mas **apareceu** uma linha com o `id` e o nome dela em `atleta_arquivo`.

```
CREATE OR REPLACE FUNCTION fn_tg_arquivar_atleta()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO atleta_arquivo (atleta_id, nome, arquivado_em)
    VALUES (OLD.id, OLD.nome, now());
    RETURN NEW;
END;
$$;

CREATE TRIGGER tg_atleta_arquivar
BEFORE DELETE ON atleta
FOR EACH ROW EXECUTE FUNCTION fn_tg_arquivar_atleta();
```

Com base no código, assinale a opção que explica corretamente o ocorrido.

A Em um `DELETE` não existe linha nova, e ler `NEW` interrompe o comando com o erro `record "new" is not assigned yet`; o `DELETE 0` mostra que esse erro foi ocultado pelo servidor.

B Em um DELETE, NEW vale NULL; devolvê-lo em um gatilho BEFORE ... FOR EACH ROW cancela a exclusão da linha sem erro. A função deve terminar com RETURN OLD.

C Em um gatilho BEFORE DELETE, OLD ainda não pode ser lido; o gatilho deveria ser AFTER DELETE, momento em que a linha apagada fica disponível em OLD.

D O INSERT em atleta_arquivo falhou e desfez a exclusão; a correção é acrescentar um bloco EXCEPTION WHEN OTHERS que ignore a falha e deixe o DELETE prosseguir.

Questão 9

Um cinema quer um relatório com **todos** os filmes em cartaz e, ao lado, as sessões de setembro de 2026; filmes ainda sem sessão em setembro devem aparecer com a data em branco. Tabelas: filme(id, titulo) e sessao(id, filme_id, data). O cinema tem 14 filmes em cartaz, 6 deles sem sessão em setembro, mas o relatório trouxe apenas os 8 que têm sessão.

```
SELECT f.titulo, s.data
FROM filme f
LEFT JOIN sessao s ON s.filme_id = f.id
WHERE s.data >= '2026-09-01' AND s.data < '2026-10-01'
ORDER BY f.titulo;
```

Considerando o cenário e a consulta, assinale a alternativa que apresenta a análise correta e a providência adequada.

A O LEFT JOIN preserva as linhas da tabela da direita, que aqui é sessao; a providência é inverter a ordem das tabelas: FROM sessao s LEFT JOIN filme f ON f.id = s.filme_id.

B A comparação entre s.data e um texto como '2026-09-01' falha para as datas sem sessão; a providência é converter as datas com CAST(s.data AS text) antes de comparar.

C O LEFT JOIN não consegue trazer filmes sem sessão quando há filtro de data; a providência é trocá-lo por INNER JOIN, que devolve todos os filmes, com ou sem sessão.

D O filtro sobre sessao está no WHERE e descarta as linhas em que s.data é nulo; a providência é levar as condições de data para a cláusula ON do LEFT JOIN.

Questão 10

Uma horta comunitária divide a colheita entre os canteiros e criou a função abaixo.

```

CREATE OR REPLACE FUNCTION fn_kg_por_canteiro(p_total_kg numeric, p_canteiros
integer)
RETURNS numeric
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN round(p_total_kg / p_canteiros, 2);
EXCEPTION
    WHEN division_by_zero THEN
        RAISE NOTICE 'Nenhum canteiro informado; devolvendo 0.';
        RETURN 0;
END;
$$;

SELECT fn_kg_por_canteiro(84, 0);

```

Considerando o contexto apresentado, avalie as asserções a seguir e a relação proposta entre elas.

I. A chamada devolve 0 e exibe o aviso `Nenhum canteiro informado; devolvendo 0.`, sem que o `SELECT` seja interrompido por erro.

PORQUE

II. Em PL/pgSQL, toda divisão por zero resulta automaticamente em `NULL`, o que dispensa o tratamento com o bloco `EXCEPTION`.

A respeito dessas asserções, assinale a opção correta.

A A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.

B As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.

C A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.

D As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.

Parte II — Questão discursiva

Questão 11 — responda à mão, no papel.

Uma agência de ecoturismo leva grupos para trilhas. Cada trilha tem um limite de pessoas por grupo, definido pelo órgão ambiental, mas o site e o atendimento por telefone têm registrado grupos maiores que o limite.

Esquema simplificado:

```
CREATE TABLE trilha (
    id          serial PRIMARY KEY,
    nome        text    NOT NULL,
    limite_pessoas integer NOT NULL CHECK (limite_pessoas > 0)
);

CREATE TABLE grupo (
    id          serial PRIMARY KEY,
    trilha_id   integer NOT NULL REFERENCES trilha (id),
    data_saida  date    NOT NULL,
    pessoas     integer NOT NULL CHECK (pessoas > 0)
);
```

Regra de negócio: a quantidade de pessoas de um grupo não pode ultrapassar o `limite_pessoas` da trilha escolhida — nem quando o grupo é registrado, nem quando a quantidade é alterada depois.

Faça o que se pede:

- Escreva, em PL/pgSQL, a **função de gatilho** que garante a regra: uma função `RETURNS trigger` que busca o limite da trilha, testa a regra com `IF`, recusa a operação com `RAISE EXCEPTION` e termina com o `RETURN` correto.
- Escreva o comando **CREATE TRIGGER** que associa a função à tabela correta, com o momento `BEFORE`, os eventos necessários, `FOR EACH ROW` e `EXECUTE FUNCTION`.
- Explique **quando** o gatilho dispara (momento `BEFORE` ou `AFTER`, evento e nível `FOR EACH ROW` ou `FOR EACH STATEMENT`) e por que essa escolha garante a regra.
- Explique o que acontece quando um comando viola a regra: o que o servidor responde a quem executou o comando, o que acontece com esse comando e o que fica gravado na tabela.

Critérios de correção

- Função de gatilho com cabeçalho correto: sem parâmetros, `RETURNS trigger`, `LANGUAGE plpgsql`, bloco `BEGIN ... END` e retorno adequado — 1,0 pt
- Lógica da regra: leitura do limite da trilha, comparação correta com `NEW` e `RAISE EXCEPTION` com mensagem clara — 1,5 pt
- `CREATE TRIGGER` correto: momento, eventos, tabela, `FOR EACH ROW` e `EXECUTE FUNCTION` — 1,0 pt
- Explicação do momento, dos eventos e do nível do gatilho, ligada à regra — 0,75 pt
- Explicação do efeito da violação: a exceção interrompe a função, o comando que disparou o gatilho é cancelado por inteiro, nada daquele comando fica gravado e quem o executou recebe a mensagem (`ERROR`, `HINT` e `CONTEXT`) — 0,75 pt